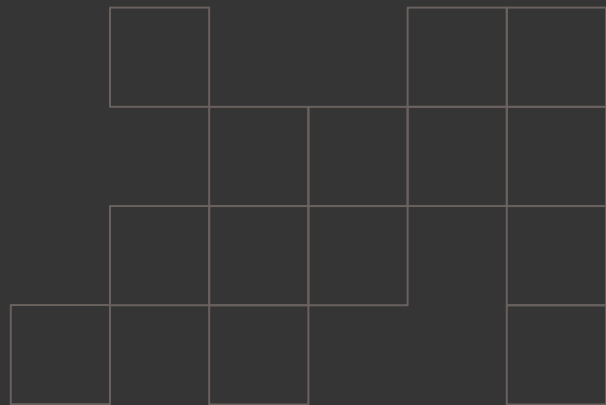


LiDAR Demo

From Rays to Reality:
Simulating Camera and LiDAR
Sensors

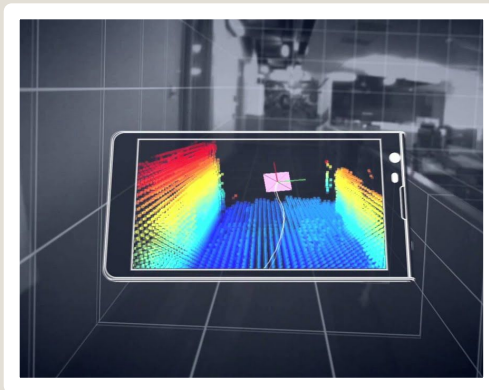
Magen, Rajas, Lyn, & Emma



Motivation

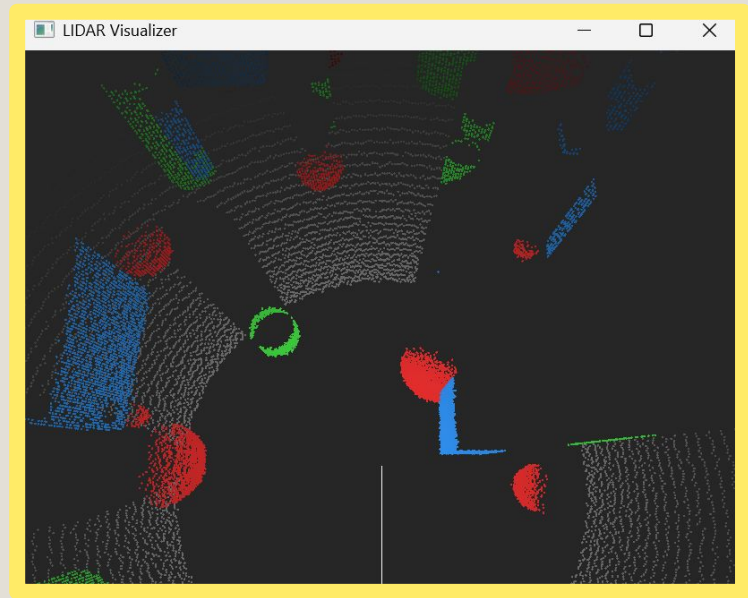
Build camera and LiDAR sensor simulations from scratch using ray-based rendering.

- Modern systems rely on sensors (cameras, LiDAR)
- Used in robotics, self-driving cars, AR/VR
- Goal: simulate these sensors from scratch using graphics



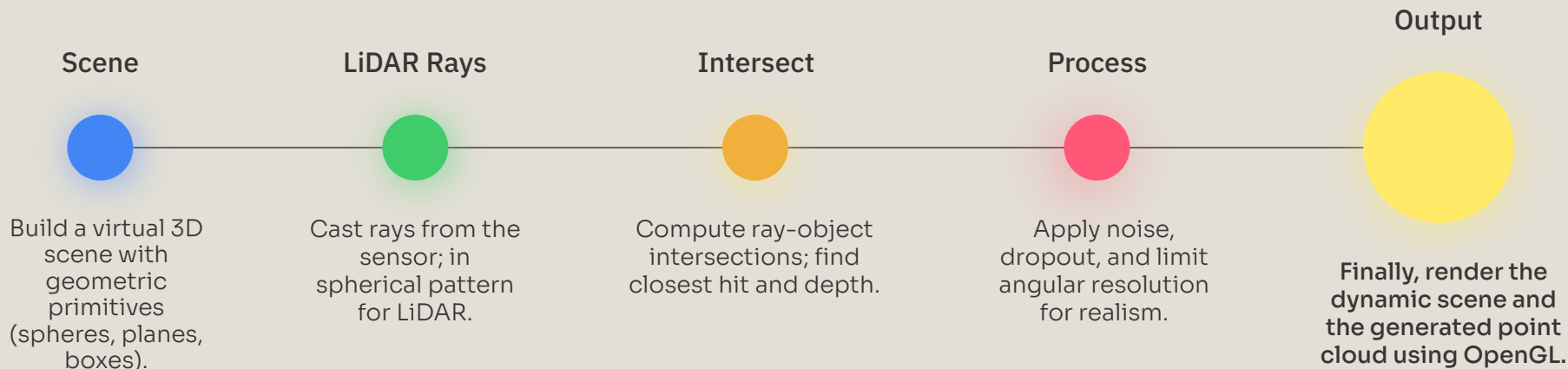
What We Built

- Ray-based rendering system
- RGB camera simulation
- Depth map generation
- LiDAR point cloud simulation
- Real-time visualization (OpenGL)

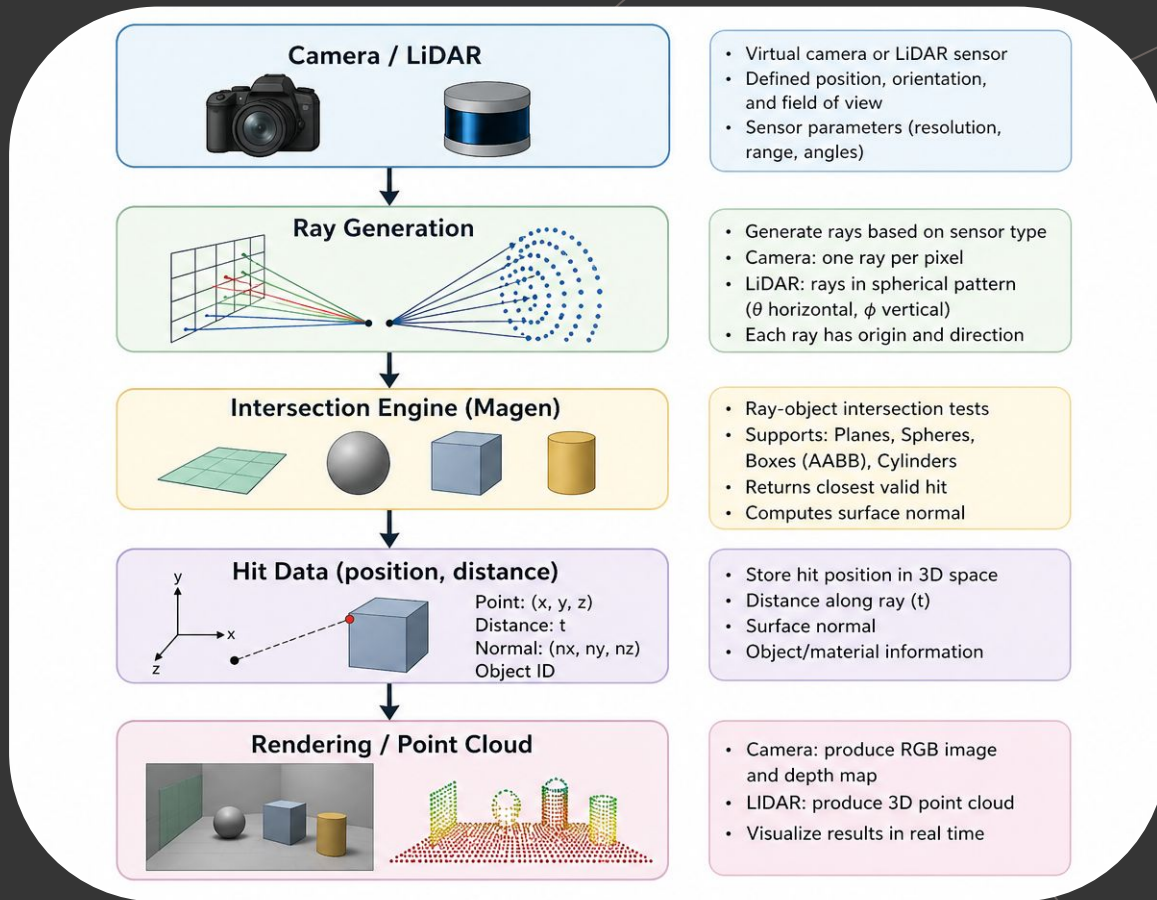


Method

Rendering Pipeline

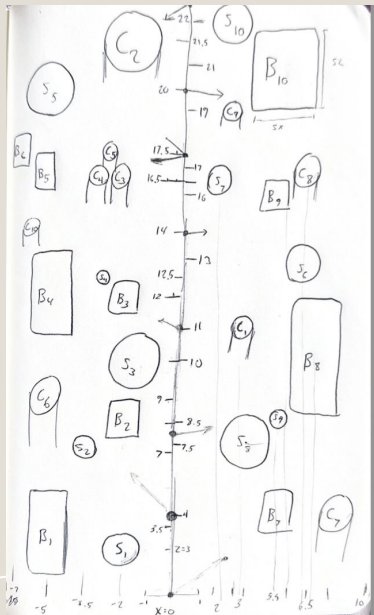


System Overview |



Scene Generation & Camera Path

Initial Sketch
of 3D Scene ->



In the Future

Move past a hardcoded scene and use procedural generation to test different scene layouts without manual placement.

Building Our Scene

Scene Generation

Created a 3D scene manually in C++ with around 30 primitives of boxes, cylinders, and spheres for the LiDAR to pick up.

Objects are along the Z-Axis from 0 to 22, the X-Axis from -8 to 8, the Y-Axis from 0 to 4.

Camera Path

Built a camera path with 12 keyframes in a Figure 8.

The camera path stays within $X=-1.5$ to $X=3$ and the Figure 8 crosses at $Z=12$.

Bezier Path Interpolation

Keyframes store both camera position and look-at target

Uses Catmull-Rom Tangents derived automatically from neighboring keyframes.

Catmull-Rom and Bezier interpolation guarantees smooth velocity through keyframes — no sudden direction changes

LiDAR Ray Generation & Sampling

Spherical Sampling:

Generated rays using spherical coordinates (θ, ϕ) to mimic spinning mechanical sensor sweeps.

$$D = \langle \cos(\phi)\sin(\theta), \sin(\phi), \cos(\phi)\cos(\theta) \rangle$$

Coordinate Mapping:

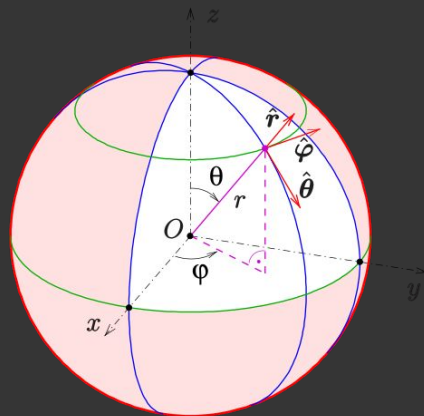
- x -component: $\cos(\phi)\sin(\theta)$
- y -component: $\sin(\phi)$
(Vertical/Elevation)
- z -component: $\cos(\phi)\cos(\theta)$

Customizable Parameters:

- Configurable Vertical Field of View (FOV) set to 40° .
- Adjustable Angular Resolution: 0.5° between rays.
- Defined maximum scanning range: 15 units.
- Rays cast per scan
 $= (360^\circ / 0.5) \times (40^\circ \text{ FOV} / 0.5)$
 $= 720 \times 80$
 $= \mathbf{57,600}$

Pipeline Integration:

Passed generated ray batches directly into the Intersection Engine to calculate precise depth and object hits.



Ray-Object Intersections

- Implementation for:

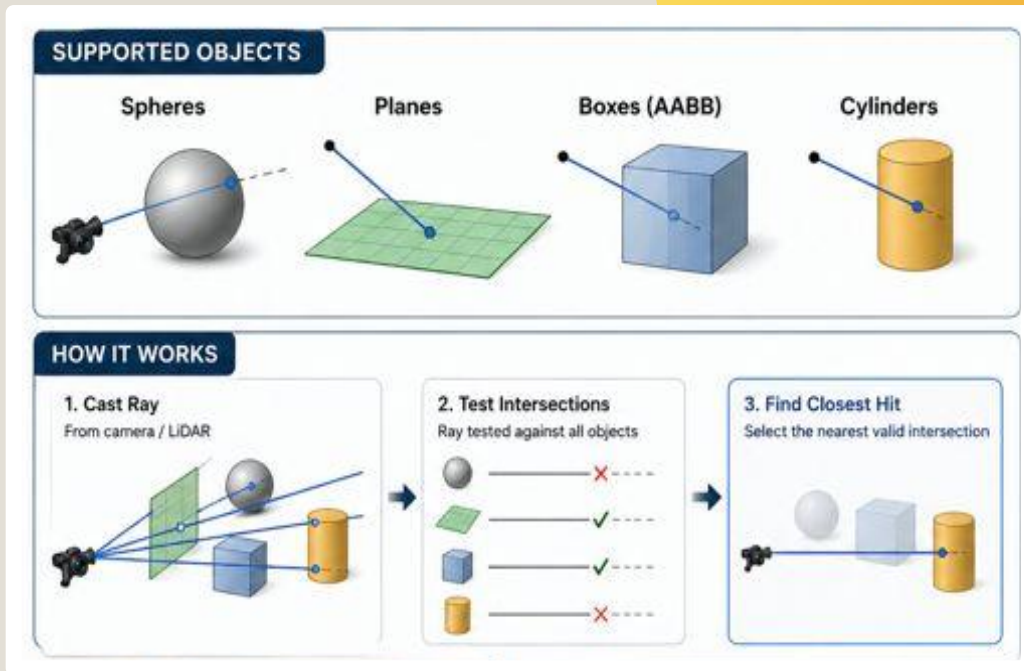
- Spheres
- Planes
- Boxes (AABB)
- Cylinders

- Computes:

- Closest hit
- Distance (t)
- Surface normal

- Handles:

- Numerical stability
- Edge cases (parallel rays, invalid t)



Point Cloud Processing

1

Realism & Sensor Imperfections

- 5cm std dev
Gaussian noise into distance measurements
- 2% random ray dropout rate
→ real-world hardware limitations

2

Semantic Segmentation

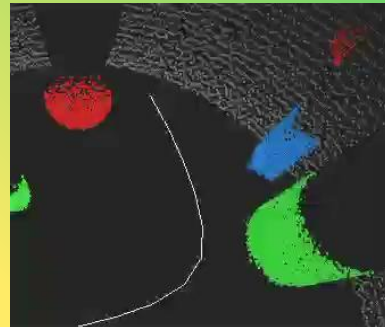
Mapped distinct base colors to specific object classes:

- Ground = Gray
- Spheres = Red
- Boxes = Blue
- Cylinders = Green

3

Depth Perception Cue

Linear fade-to-black intensity multiplier based on hit distance to naturally enhance 3D depth perception against the dark OpenGL background

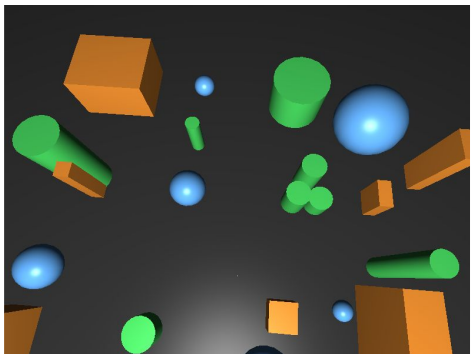


OpenGL Rendering

This part only deals with the visualization of the raw scene data and the calculated point cloud data.

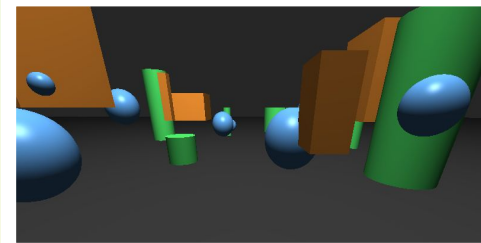
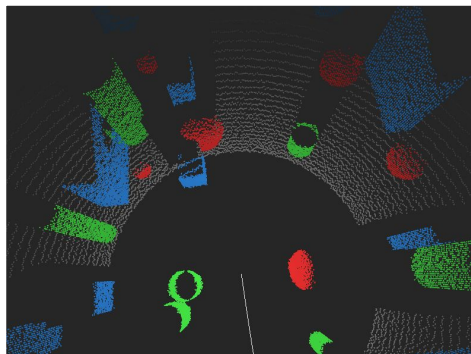
It features:

- Phong-shader for a realistic raw scene
- Point Cloud shader for LiDAR point cloud
- Camera view and projection matrices for perspective rendering
- A rendering loop to update the camera and sensor positions and scan the scene every frame
- Switch between two cameras to visualize from the top-view or the LiDAR sensor view



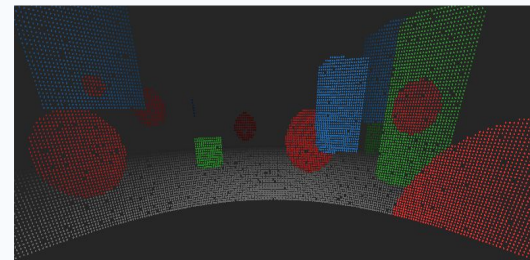
Camera Top PoV

LiDAR Top PoV

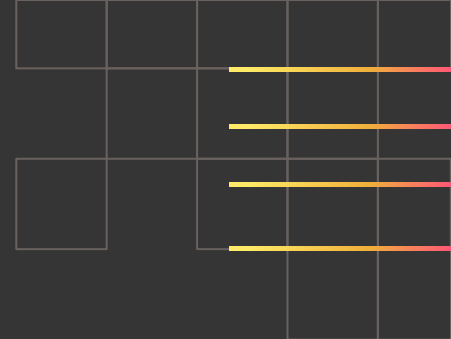
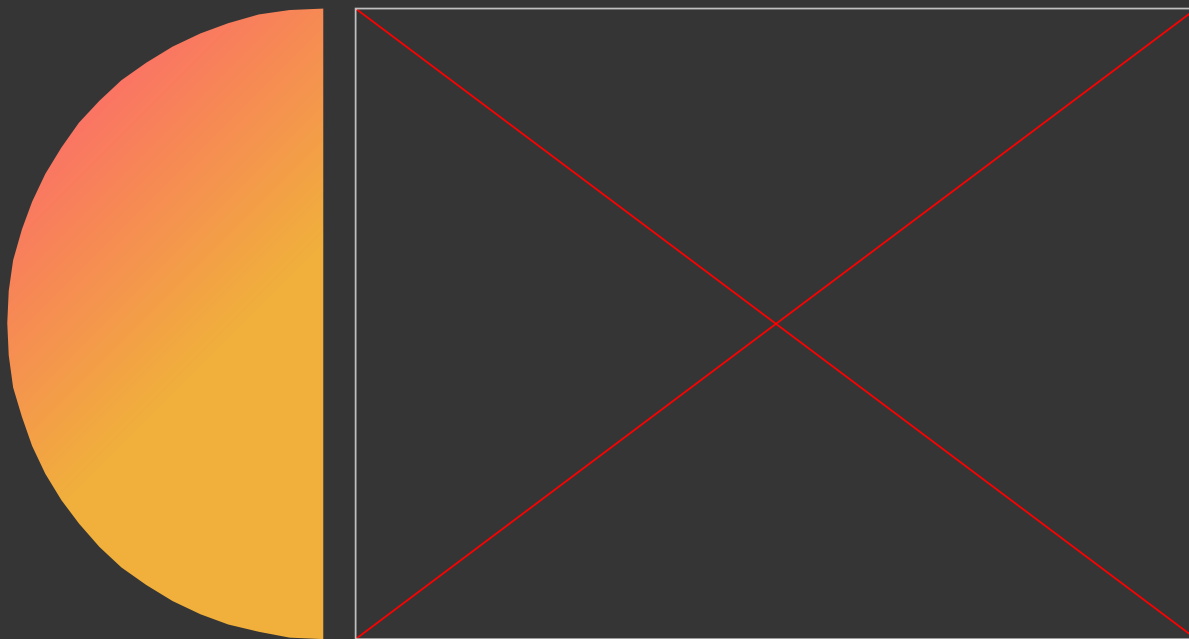


Camera PoV

LiDAR PoV



Final result



Conclusion and Future Scope

OUR WORK

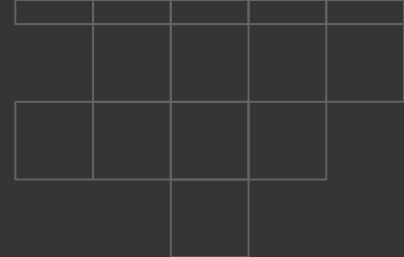
We have a working LiDAR scan simulator for any given path and primitive scene.

The scene updates with low-latency.

FUTURE STEPS

- Dynamic scene updation
 - Loading custom meshes in the scene
 - Ray intersection code for custom mesh file
 - Moving objects in the scene
-

Contributions



Team

Emma Jerrier

ejerrier@usc.edu

Lyn El Sayed Kassem

elsayedk@usc.edu

Magen Mozeh

mozeh@usc.edu

Rajas Chitale

rachital@usc.edu

Tasks

Building the hierarchical scene, Object placement, Bezier curve interpolations for camera path, Spherical sampling..

LiDAR sensor ray generation, Point cloud sampling and processing (Gaussian noise, semantic color assignment, depth cues).

Ray-Object intersection engine, Multi-object support, Scene integration, Edge case handling.

OpenGL setup, OpenGL camera/image rendering, Shader management (Phong vs. Point Cloud).



Thank you

Any questions?

FROM RAYS TO REALITY
SPRING 2026